

Specification of team software project

Department of Software Engineering

Faculty of Mathematics and Physics, Charles University

Solver: Miroslav Štochel

Study program: Informatika – Softwarové a datové inženýrství

Project title: Rozšíření nástroje ORMorpher

Project type: Research project

Supervisor: Ing. Pavel Koupil, Ph.D.

Consultants: doc. RNDr. Irena Holubová, Ph.D., RNDr. Michal Kopecký, Ph.D.

1 Úvod

V oblasti databázových systémů existuje řada nástrojů a přístupů pro migraci dat, evoluci databázového schématu a propagaci změn do souvisejících datových struktur a dotazů. Výrazně omezenější je však podpora obdobných změn na úrovni aplikačního kódu. Změna databázové technologie, objektově-relačního mapovacího frameworku nebo způsobu perzistence proto obvykle vyžaduje ruční úpravy entit, mapování, dotazů a dalšího kódu datové vrstvy aplikace.

Výzkumná skupina Adaptive Data Management (ADaM) Research Group [1] se mimo jiné zabývá automatizací migrace a transformace datových a aplikačních artefaktů napříč různými technologiemi. V rámci tohoto výzkumu vzniká nástroj ORMorpher [3], jehož původní implementaci vytvořil Milan Abrahám ve své diplomové práci [4] jako proof of concept pro ověření vybraných principů překladu mezi ORM a obdobnými frameworky.

ORMorpher je založen na mezirepresentaci nezávislé na konkrétním frameworku: parser zdrojového frameworku převádí podporované artefakty do mezirepresentace a builder cílového frameworku z ní generuje odpovídající artefakty. Tento přístup omezuje potřebu vytvářet samostatné párové překladače. Současná implementace však podporuje pouze vybrané konstrukce frameworků Dapper, NHibernate a EF Core a jednotlivé směry překladu nejsou implementovány ve stejném rozsahu. Součástí proof of concept je rovněž Advisor, který experimentálně porovnává vybrané frameworky a umožňuje optimalizační výběr na základě měřených charakteristik.

Cílem projektu je existující proof of concept systematicky rozšířit, nikoli z něj vytvořit produkční nástroj. Projekt doplní složitější mapovací konstrukce, využití metadat z databázového schématu a podporu Java frameworků Hibernate, MyBatis a EclipseLink. Zásadním cílem je tím ověřit použitelnost společné mezirepresentace a překladového přístupu napříč různými programovacími jazyky a technologickými ekosystémy. Součástí projektu je rovněž testovací a běhové prostředí pro automatizované ověřování a experimenty, které umožní posoudit praktickou použitelnost přístupu a připravit srovnání s metodami založenými na velkých jazykových modelech.

Cílem projektu není úplná podpora všech konstrukcí jednotlivých frameworků. Překlad je omezen na explicitně vymezenou společnou podmnožinu konstrukcí, kterou dokáže zachytit mezirepresentace a pro kterou lze ověřit odpovídající chování napříč .NET a Java ekosystémem.

1.1 Vztah k dalším projektům

Projekt přímo navazuje na diplomovou práci Milana Abraháma a její prototyp [4]. Dále rozvíjí návrh mezireprezentace, implementaci překladové pipeline a experimentální infrastrukturu původního řešení.

Tematicky souvisí také s nástrojem mm-mapsearch [2], který hledá vhodnou reprezentaci dat s ohledem na zadaný workload, zejména strukturu a charakter dotazů. Zatímco mm-mapsearch řeší volbu reprezentace především na úrovni dat a databázového systému, ORMorpher tuto problematiku doplňuje na úrovni aplikační datové vrstvy transformací entit, mapování a dotazů mezi jednotlivými frameworky.

Další související směr představuje diplomová práce Martina Čorovčáka, která navrhuje univerzální objektový model (UOM) a migraci aplikačního kódu mezi relačními, grafovými a dokumentovými databázovými systémy, včetně mapování ORM, OGM a ODM, s využitím velkých jazykových modelů. Tento přístup je obecnější, ale současně může být nedeterministický a výpočetně i finančně nákladnější. ORMorpher oproti tomu zkoumá deterministický překlad založený na explicitních pravidlech, heuristikách a mezireprezentaci.

S projektem souvisí rovněž diplomová práce Ivony Oboňové zaměřená na migraci dotazů mezi relačními a grafovými databázovými systémy s využitím velkých jazykových modelů. Tyto práce společně pokrývají komplementární části problému: volbu datové reprezentace, transformaci aplikačního mapování a migraci dotazů mezi různými databázovými modely a technologiemi.

Výzkumným přínosem tohoto projektu je především ověření cross-language použitelnosti ORMorpheru rozšířením o Java frameworky, doplnění proof of concept implementace směrem k prakticky použitelnému experimentálnímu nástroji a příprava reprodukovatelného experimentálního prostředí. To poskytne deterministickou baseline pro následné porovnání s LLM-based metodami, včetně variant s RAG nebo agentním přístupem. Výsledky projektu mají sloužit jako podklad pro vědeckou publikaci, přednostně v odborném vědeckém časopise; plnější experimentální porovnání s LLM bude dále rozvíjeno v navazující diplomové práci řešitele (viz kapitolu 4).

2 Popis projektu

Nástroj (v repozitáři ORMConverter) je backendové řešení v .NET 10: REST API v ASP.NET Core s jednoduchým frontendem. Překlad probíhá ve třech fázích: *parser* zdrojového frameworku převede vstupní artefakty do mezireprezentace, nepovinná fáze *doplnění* dotáhne chybějící mapovací fakta z katalogu připojené databáze a *builder* cílového frameworku vygeneruje výstupní artefakty. Vedle artefaktů vrací každý převod strukturovanou diagnostiku: záznamy o ztrátách (fakt, který cíl nevyjádří), konvencích (výstup tvrdí něco, co zdroj neřekl), konfliktech (dva zdroje tvrdí totéž jinak), neúplnostech, selháních a faktech doplněných z katalogu (výstup tvrdí něco, co dodala databáze, ne zdroj). Dotaz, který by v cíli vrátil jinou množinu řádků, se zásadně nevydává – vadný překlad končí záznamem, ne tichou sémantickou změnou. Referenční databází je SQL Server se vzorovou databází WideWorldImporters; celé prostředí včetně testovací sady běží v Dockeru.

2.1 Výchozí stav při převzetí

Snímek stavu k 15. 7. 2026, zdokumentovaný v repozitáři jako zmražený referenční bod. Prototyp uměl: překlad entit a mapování mezi třemi .NET ORM v libovolném směru, překlad dotazů jedním směrem (EF Core LINQ na Dapper SQL), ILP Advisor pro Dapper a EF Core a REST API s frontendem. Chybělo: kompozitní primární klíče (model nesl jen jednosloupcový klíč bez pořadí a strategie), vícesloupcové cizí klíče, jakékoli čtení databázového katalogu, priority zdrojů metadat a hlášení konfliktů, validace úplnosti (nepodporované konstrukce se potichu vynechávaly), dotazové parsery pro Dapper a NHibernate, dotazové buildery pro EF Core a NHibernate, podmínky jiné než plochá konjunkce, záznam běhu a jakákoli podpora javovského ekosystému. Typový model byl vázaný na CLR, takže pro javovské frameworky nepoužitelný.

2.2 Stav verze 1.2.0

Práce je rozdělena do tří cílů. **Cíl 1 – rozšíření a dotažení tří .NET frameworků – je hotový** a vydáný jako verze 1.2.0. Nástroj ve verzi 1.2.0 poskytuje:

- **Složené identifikátory a vztahy** (F1–F3): mezireprezentace nese klíč jako seřazený seznam částí s per-part strategií generování a kanonickými parametry generátorů, vztahy s uspořádanými páry sloupců; 1:1, 1:N i N:M (přes explicitní spojovací entitu) mají uložení a načtení spuštěné proti databázi v obou cílových frameworkích.
- **Doplňování z databázového katalogu** (F4–F6): jediná komponenta čte metadata z katalogu SQL Serveru (sloupce, typy, nullabilitu, klíče, unikátní omezení, identity) dávkově; doplní fakta, která zdroj neuvedl, rozpor hlásí konfliktem se zdrojovou hodnotou. Z entity, která má jen názvy a jazykové typy (typický Dapper vstup), vznikne úplné kompilovatelné mapování včetně vztahů odvozených z cizích klíčů a detekce spojovacích tabulek.
- **Úplná dotazová matice** ve všech devíti směrech mezi třemi frameworky, v rozsahu projekce, filtrace (podmínkový strom s AND/OR/NOT a závorkováním), joiny, agregace, řazení, stránkování, poddotazy v podmínce a množinové operace; NHibernate se čte i z holého HQL vlastním parserem.
- **Validaci a diagnostiku** (F11): brána úplnosti před generováním, šest druhů diagnostických záznamů, záznam běhu s identifikátorem a verzemi (S6). Vícesouborový vstup a výstup po souborech (část F14).
- **Systémové požadavky**: modularita (S1), determinismus (S2), měřený výkon překladu (S3: testovací projekt o 100 entitách a 100 dotazech se přeloží méně než sekundu), kontejnerovou reprodukovatelnost (S5 – systém i testovací sada jedním příkazem), pozorovatelnost (S6) a základní scénář rozhraní v pěti krocích (S7).

Testovací sada má **615 testů** (xUnit v3, 0 přeskočených při připojení databázi) s měřeným pokrytím 75,5 % řádků; běží v CI při každém push a pull requestu. Generované artefakty se ověřují na čtyřech stupních: tvar (řetězcové aserce), kompilovatelnost (kompilace Roslynem, validace XML proti XSD), přijetí frameworkem (stavba session factory NHibernate, validace modelu EF Core, parsování SQL) a běh proti databázi (uložení a načtení entity vygenerovaným kódem). Hranice záruk je vyslovená: Advisor s benchmarkingem je z nároku verze vyňatý (nemá testy, měří dva frameworky ze tří), stejně jako dědičnost a komponenty v NHibernate mapování, cílové dialekty mimo SQL Server a celý javovský ekosystém.

2.3 Zbývající práce a postup

Cíl 2 – Java frameworky a cross-language ověření (F7–F10, F12–F13) je jádrem rozšíření ze zadání: parser a builder pro Hibernate, MyBatis a EclipseLink, překlad mezi .NET a Javou a testovací infrastruktura pro Javu s diferenčním ověřením výsledků dotazů. Tento cíl má především ověřit, že společná mezireprezentace a překladový přístup nejsou vázané na jediný programovací jazyk nebo technologický ekosystém. Slovníkové předpoklady jsou hotové – typový model mezireprezentace je jazykově neutrální na jazykové i databázové straně a parametry generátorů klíčů se nesou kanonicky. Prvním krokem cíle je rozhodnutí, kudy javová strana do řešení vstupuje (viz kapitulu 5).

Cíl 3 – Advisor, benchmarking a experimenty (F15, T1–T7) naváže: rozšíření Advisoru o nové frameworky, izolace spouštění měřeného kódu (S4) a experimentální vyhodnocení podle kapitoly 4. Experimenty mají ověřit praktickou použitelnost rozšířeného proof of conceptu a připravit reprodukovatelnou baseline pro srovnání s LLM-based přístupy.

Postup práce je dokumentovaný přímo v repozitáři: každá netriviální volba vzniká jako číslované rozhodnutí se zvažovanými variantami a důsledky, aktuální chování popisuje průběžně udržovaný architektonický dokument, zbývající práce jediný seznam otevřených položek a hranici záruk každé vydání vyslovuje explicitně.

2.4 Funkční požadavky

F1 – Reprezentace komplexních identifikátorů v mezireprezentaci. Entita může mít identifikátor tvořený jednou nebo více vlastnostmi. Mezireprezentace zachová pořadí částí klíče, jejich datové typy, názvy sloupců a strategii generování.

F2 – Překlad komplexních identifikátorů ve všech podporovaných frameworkích. Složené primární klíče se parsují i generují ve všech podporovaných .NET a Java frameworkích mechanismem daného frameworku (více klíčových sloupců, @EmbeddedId, @IdClass, XML mapování); výstup se zkompile a obsahuje všechny části klíče.

F3 – Cizí klíče odkazující na složené identifikátory. Překlad vztahů s vícesloupcovým cizím klíčem, včetně 1:1, 1:N a N:M přes spojovací tabulku; každý typ vztahu má zkompileovaný a spuštěný test.

F4 – Načtení metadat z databázového schématu. Neobsahuje-li vstupní kód úplná mapovací metadata, systém načte z připojené databáze alespoň tabulky, sloupce, datové typy, nullabilitu, primární a cizí klíče, unikátní omezení, délku, přesnost a měřítko.

F5 – Sloučení metadat z více zdrojů. Informace ze zdrojového kódu, anotací, externích mapovacích souborů a databázového katalogu se slučují do jediné mezireprezentace s dokumentovanou prioritou zdrojů a hlášením konfliktů (nullabilita, název sloupce, datový typ, primární klíč).

F6 – Generování úplného mapování z neúplného vstupu. Z frameworku s neúplnou či implicitní mapovací informací (Dapper, MyBatis) vznikne doplněním z databáze úplné mapování cílového frameworku: z entity nesoucí jen názvy a jazykové typy vlastností vznikne kompilovatelná cílová entita se správnými názvy tabulek a sloupců, klíči a omezeními.

F7 – Podpora Hibernate. Parser a generátor pro podporovanou podmnožinu: entity, tabulky a schémata,

sloupce, jednoduché a složené klíče, základní vztahy a read-only dotazy alespoň v JPQL nebo HQL. Testovací sada pokryje všechny implementované kategorie konstrukcí a generované projekty se sestaví.

F8 – Podpora MyBatis. Parser a generátor pro podporovanou podmnožinu: Java třídy, XML nebo anotované mapování, resultMap, parametry, dynamicky parametrizované read-only dotazy a ručně psané SQL joiny. Testovací sada pokryje implementované kategorie mapování i vícetabulkové dotazy.

F9 – Podpora EclipseLink. Parser a generátor pro EclipseLink/JPA v rozsahu společné mezireprezentace: anotované entity, složené klíče, vztahy a JPQL dotazy. Testovací sada pokryje implementované kategorie konstrukcí včetně překladu mezi Hibernate a EclipseLink.

F10 – Cross-language překlad mezi .NET a Javou. Překlad entity, mapování a podporovaného read-only dotazu mezi implementovanými .NET a Java frameworky v rozsahu společné podporované podmnožiny. Testovací matice bude obsahovat reprezentativní end-to-end scénáře pro směry .NET→Java, Java→.NET a Java→Java.

F11 – Validace cílových artefaktů. Před předáním výsledku se ověří strukturální úplnost mezireprezentace, omezení cílového frameworku a syntaktická správnost generovaných souborů. Nepodporované konstrukce se nesmí potichu vynechat; každý neúspěšný překlad vrátí strukturovanou diagnostiku (framework, artefakt, chybějící vlastnost, důvod selhání).

F12 – Spustitelná testovací sada pro Java frameworky. Automatizovaná sada sestaví a spustí vygenerovaný kód Hibernate, MyBatis a EclipseLink nad stejnou databází a daty; je spustitelná jedním příkazem a obsahuje dostatečný počet jednotkových a integračních testů pokrývajících všechny podporované kategorie konstrukcí.

F13 – Diferenční ověření výsledků dotazů. Zdrojová i přeložená varianta dotazu se spustí a porovnají se normalizované výsledky: pořadí se zohlední jen je-li dané dotazem, práce s numerickou přesností a hodnotami null je konfigurovatelná. Testovací sada bude obsahovat reprezentativní dvojice dotazů pro podporované kategorie; součástí bude také negativní test se záměrně chybným překladem, který musí být odhalen.

F14 – Překlad celých tříd a dávkových vstupů v UI. Vložení či nahrání jedné nebo více celých tříd, případně archivu projektu; volba zdrojového a cílového frameworku; validace a překlad. Rozhraní zobrazí vstup, mezireprezentaci, výstup a diagnostiku po jednotlivých souborech; scénář zpracuje projekt s alespoň deseti entitami a pěti dotazy bez ručního zadávání každého artefaktu.

F15 – Výběr cílového frameworku a optimalizace (sekundární cíl). Uživatel může přímo zvolit cílový framework; podle časových možností bude existující Advisor rozšířen také o nové frameworky a umožní doporučení na základě měřeného času, paměti a vah dotazů. Tato funkcionalita je doplňková vůči hlavnímu cíli cross-language překladu a jeho ověření.

2.5 Nefunkční požadavky

S1 – Modulární rozšiřitelnost. Nový framework se přidává novým parserem a builderem přes definovaná rozhraní, bez zásahu do ostatních frameworků a bez párových překladačů.

S2 – Determinismus. Stejný vstup, konfigurace, schéma a verze nástroje dávají shodné artefakty ve 100 % běhů.

S3 – Výkon překladu. Překlad musí být prakticky použitelný pro reprezentativní projektovou testovací sadu; čas samotného překladu a čas načtení databázových metadat se měří a reportují odděleně. Konkrétní výkonnostní

výsledky budou součástí vyhodnocení, nikoli pevnou funkční hranicí.

S4 – Izolace a bezpečnost spouštění. Cizí a generovaný kód v izolovaném prostředí s limity; žádné přihlašovací údaje v artefaktech a ložích.

S5 – Přenositelné a reprodukovatelné prostředí. Systém, databáze, testovací projekty a experimentální pipeline v kontejnerové konfiguraci; reprodukce testů jedním příkazem.

S6 – Pozorovatelnost a auditovatelnost. Každý překlad má identifikátor běhu a strojově čitelný záznam: zdroje metadat, varování, chyby, verze frameworků i nástroje.

S7 – Uživatelská přívětivost. Validace před spuštěním, průběžný stav, chyby na úrovni souboru a řádku, stažení výstupu; základní scénář nejvýše v pěti krocích.

2.6 Experimentální požadavky

Experimentální část je rozdělena na požadavky tohoto projektu a navazující experimenty, které budou plně rozvedeny v diplomové práci řešitele. Projekt musí připravit reprodukovatelné testovací prostředí, základní cross-language experimenty a měřicí infrastrukturu; rozsáhlejší LLM experimenty nejsou podmínkou dokončení tohoto projektu.

T1 – Reálná případová studie. Projekt připraví alespoň jednu reprezentativní případovou studii nebo testovací sadu s netriviálními entitami, vztahy a read-only dotazy. Rozšíření na více reálných open-source aplikací je předmětem navazující diplomové práce.

T2 – Matice překladů. Projekt ověří směry .NET→Java, Java→.NET a Java→Java na reprezentativních kategoriích dotazů a mapovacích konstrukcí, zejména projekci, filtraci, joinu, agregaci, stránkování, poddotazech a množinových operacích.

T3 – Metriky korektnosti. Experimentální infrastruktura bude měřit alespoň podíl parsovatelných, kompilovatelných a spustitelných výstupů, funkční ekvivalenci a úplnost přenesených podporovaných mapovacích vlastností.

T4 – Příprava LLM baseline. Projekt připraví testovací data, rozhraní a měřicí infrastrukturu tak, aby bylo možné stejnou sadu vstupů porovnat s překladem pomocí velkého jazykového modelu. Úplné zero-shot a few-shot experimenty jsou předmětem navazující diplomové práce.

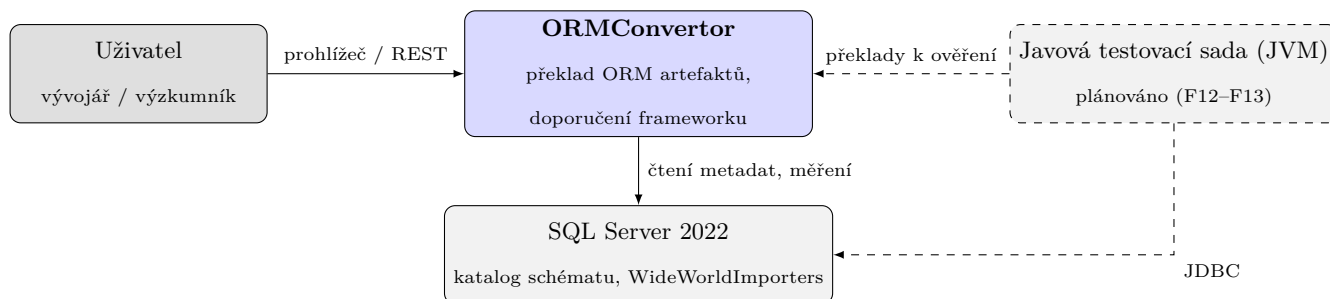
T5 – Příprava RAG / agentní varianty. Projekt připraví podklady a experimentální rozhraní umožňující v navazující práci doplnit dokumentaci cílového frameworku, informace o schématu a relevantní příklady. Samotné rozsáhlé porovnání RAG a agentních variant není podmínkou dokončení projektu.

T6 – Příprava ablace. Návrh testovacího prostředí musí umožnit v navazující práci samostatně vyhodnotit vliv databázových metadat, validace, opravného cyklu a případně RAG. Úplná ablační studie není požadavkem tohoto projektu.

T7 – Základní porovnání Advisoru. Existující Advisor bude podle časových možností rozšířen o nové frameworky a základně porovnán s jednotlivými frameworky a alespoň jednou jednoduchou heuristikou. Jde o sekundární experimentální cíl; nesmí omezit dokončení hlavního cross-language cíle F7–F13.

2.7 Architektura

Architekturu popisujeme prvními dvěma úrovněmi modelu C4. Na úrovni kontextu (obr. 1) je nástroj jediný systém: uživatel (vývojář migrující aplikaci, nebo výzkumník spouštějící experimenty) s ním pracuje přes prohlížeč nebo přes REST API, systém čte metadata z katalogu připojené databáze SQL Server a proti téže databázi měří výkon generovaného kódu. Javovské běhové prostředí je oddělená součást v kontejneru: testovací sada podle F12–F13 si generované artefakty bere z běžící instance přes REST a spouští je proti téže databázi.



Obrázek 1: Úroveň kontextu (C4, L1).

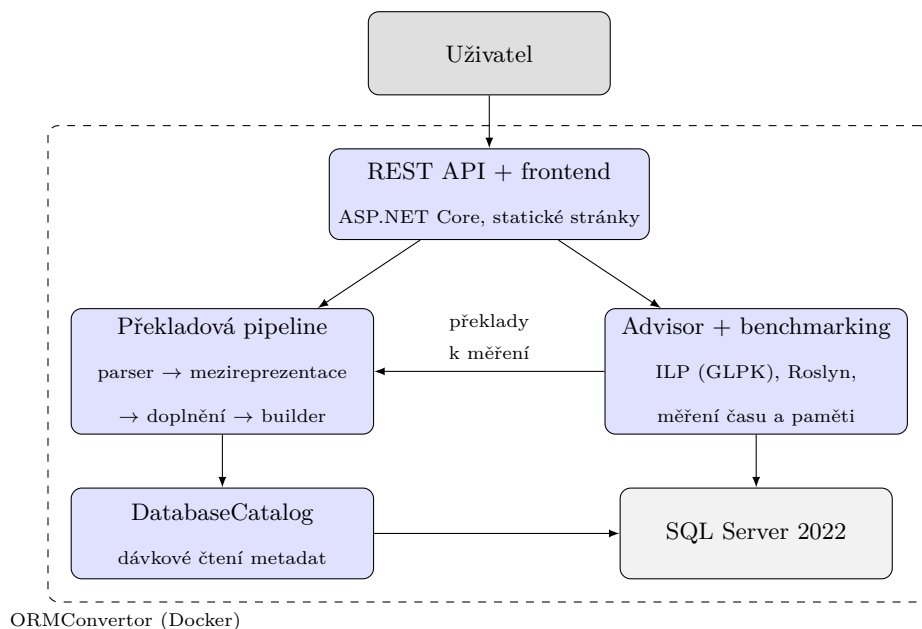
Na úrovni kontejnerů (obr. 2) je systém REST API (ASP.NET Core), překladová pipeline, čtení databázového katalogu a Advisor s benchmarkingem. Pipeline jsou vzory Adapter (wrappery frameworků), Visitor (generování dotazů z instrukcí), Builder a Factory; orchestrace pracuje s wrappery výhradně přes rozhraní. Nosné projekty jsou:

- **Model** – mezirepresentace: entity a mapování (**EntityMap**, seřazený složený klíč, vztahy s páry sloupců, unikátní omezení) a dotazové instrukce s podmínkovým stromem;
- **AbstractWrappers** – rozhraní parserů, abstraktní buildery se šablonovou metodou, deskriptory cílových frameworků (co framework vyžaduje a co nevyjádří) a typy diagnostických záznamů;
- **DapperWrappers**, **EFCoreWrappers**, **NHibernateWrappers** – po jednom projektu na framework; sdílené čtení C# entit a LINQ dotazů nesou projekty **CSharpEntityParsing** a **LinqParsing**;
- **DatabaseCatalog** – jediné místo čtení databázových metadat a fáze doplnění; wrappery na něm nezávisí;
- **OrmConverter** – orchestrace překladu, **ORMConverterAPI** – REST API a frontend, **Tests** – testovací sada;
- **Advisor** (ILP model v C nad knihovnou GLPK, volaný přes P/Invoke) a **AdvisorBenchmarking** (kompilace generovaného kódu Roslynem a měření běhu a paměti).

Pro každý javovský framework vznikne projekt s novým wrapperem. Provedenou analýzou a popsáním rozhodnutím může implementace rozšířit sdílenou vrstvu.

2.8 Uživatelské rozhraní

Rozhraní je záměrně jednoduché: statické stránky se společnou hlavičkou – rozcestník, překlad, Advisor a výkladová stránka se živými příklady. Základní scénář (S7) je jediná stránka s pěti očíslovanými kroky viditelnými najednou, takže se uživatel při opravě chyby vrací ke vstupu bez přepínání obrazovek. Makety na obr. 3–5 ukazují schematickou podobu obrazovek; obsah polí je ilustrační.



Obrázek 2: Úroveň kontejnerů (C4, L2). Javovské kontejnery (F12–F13) přibudou v cíli 2.

Vstup překladu (obr. 3). Kroky 1 a 2 volí zdrojový a cílový framework ze všech podporovaných; nabídka nese verzi z deskriptoru frameworku, tutéž, kterou pak uvádí záznam běhu (S6). Krok 3 skládá dávkový vstup podle F14 jako seznam pojmenovaných jednotek: jednotka má jméno, typ obsahu a obsah a vzniká nahráním jednoho či více celých souborů (typ se předvyplní podle přípony), přidáním prázdné jednotky a vepsáním, nebo naplněním vzorky. Nabídka typů je omezená na to, co zdrojový framework čte. Krok 4 nejprve validuje na klientovi – prázdný seznam, prázdná jednotka, typ mimo nabídku, nesprávně vytvořené XML s číslem řádku – a chybu ukáže u dotčené jednotky; syntaktické chyby zdrojového kódu hlásí server a rozhraní zobrazí jeho hlášku doslova, u SQL s řádkem a sloupcem. Po dobu volání je tlačítko zablokované a průběžný stav se hlásí pod ním (S7).

Výsledek překladu (obr. 4). Hlavička nese záznam běhu (S6): zdroj → cíl s verzemi, stav připojení ke katalogu (nenakonfigurováno, nepoužito, přečteno s časem čtení, nedosažitelné), verzi nástroje a identifikátor běhu. Artefakty stojí po souborech v panelech se zvýrazněnou syntaxí; kompletní výstup balí server do ZIP (S7). Diagnostické záznamy (F11) stojí pod výstupem: souhrn počtů podle druhu slouží zároveň jako filtr a tabulka nese druh, entitu, vlastnost, vstupní jednotku, cílový artefakt, předmět a důvod; záznamy druhu *Failure* stojí první, protože znamenají, že artefakty entity chybí. Jména souborů výstupu odvozuje klient z obsahu artefaktu.

Advisor (obr. 5) sdílí tvar s překladem: zdrojový framework, entitní jednotky, dotazy s vahami (váha je relativní četnost dotazu), omezení – limit paměti, strop počtu vybíraných frameworků a podmnožina kandidátů – a spuštění, které pro každý dotaz a každého kandidáta překládá, kompiluje a měří, a proto trvá minuty. Výsledek ukazuje srovnání: pro každý dotaz všechny změřené kandidáty seřazené podle času, s vyznačeným přiřazením, poměrem k nejrychlejšímu a součty za přiřazené řádky. Přiřazený kandidát nemusí být nejrychlejší, protože limit paměti a strop počtu frameworků jsou součástí modelu – v ukázce je strop jeden framework, takže druhý dotaz dostal Dapper, ač je v EF Core rychlejší. Vedle srovnání stojí tabulka variant podle F15: pro každého kandidáta varianta „vše v jednom frameworku“, heuristika volící pro každý dotaz nejrychlejšího kandidáta a ILP nad shodným workloadem.

ORMConvertor
Translation
Advisor
Examples

Translation

1. Source framework
Entity Framework Core 10.0.10

2. Target framework
NHibernate 5.7.0
Dapper 2.1.79
NHibernate 5.7.0
Entity Framework Core 10.0.10
Hibernate ORM 7.4.5
MyBatis 3.5.19
EclipseLink 5.0.0

3. Input files
Add files...
Add empty unit
Load sample set

Customer.cs
C# entity

```
[Table("Customers", Schema = "Sales")]
public class Customer
{
    [Key] public int CustomerID { get; set; }
    [MaxLength(100)] public string CustomerName { get; set; } = null!;
```

CustomerByCity.cs
C# LINQ query

```
var customers = context.Customers.Where(c => c.DeliveryCityID == cityId)
    .OrderBy(c => c.CustomerName).Select(c => new { c.CustomerID, c.CustomerName });
```

Order.cs
C# entity

The unit has no content.

4. Convert
Convert

Obrázek 3: Překladařová obrazovka, kroky 1–4: volba frameworků, dávkový vstup po pojmenovaných jednotkách a validace před odesláním.

5. Result
Entity Framework Core 10.0.10 → NHibernate 5.7.0
catalog read (38 ms)
ORMConvertor 3.0.0 run 2f8c1e7a...

Download all as ZIP

Customer.cs
C# entity

```
public class Customer
{
    public virtual int CustomerID { get; set; }
    public virtual string CustomerName { get; set; } = null!;
    public virtual City DeliveryCity { get; set; } = null!;
```

Customer.hbm.xml
XML mapping

```
<class name="Customer" table="Customers" schema="Sales">
  <id name="CustomerID" column="CustomerID">
    <generator class="identity"/>
  </id>
  <property name="CustomerName" length="100" not-null="true"/>
  <many-to-one name="DeliveryCity" column="DeliveryCityID"/>
</class>
```

CustomerByCity.hql
HQL query

```
select c.CustomerID, c.CustomerName
from Customer c
where c.DeliveryCity.CityID = :cityId
order by c.CustomerName
```

Intermediate representation
Customer

```
EntityMap Customer -> Sales.Customers
key CustomerID : int, generated (identity)
property CustomerName : string(100), not null
relation DeliveryCity -> City, N:1, owning
DeliveryCityID -> CityID
```

Records
Convention × 1
Supplied × 1

Kind	Entity	Property	Unit	Artifact	Subject	Reason
Convention	Customer	CustomerID	Customer.cs	Customer.hbm.xml	key generation	no strategy stated; identity assumed for an integer key
Supplied	Customer	CustomerName	Customer.cs	Customer.hbm.xml	nullability	not-null read from the catalog (Sales.Customers)

Obrázek 4: Překladařová obrazovka, krok 5: záznam běhu se stavem katalogu, artefakty po souborech, mezireprezentace a diagnostické záznamy po jednotkách.

ORMConvector
Translation
Advisor
Examples

Advisor

1. Source framework

Entity Framework Core 10.0.10

2. Entities

Load samples

Customer.cs
C# entity

[Table("Customers", Schema = "Sales")]
public class Customer { ... }

Order.cs
C# entity

[Table("Orders", Schema = "Sales")]
public class Order { ... }

3. Queries and weights

Add query

CustomerByCity.cs
weight 10

context.Customers.Where(c => c.DeliveryCityID == cityId).Select(c => c.CustomerName)

OrdersOfCustomer.cs
weight 3

context.Orders.Where(o => o.CustomerID == id).OrderByDescending(o => o.OrderDate).Take(20)

4. Constraints

Memory limit (KB)
0

Maximum frameworks to select
1

Candidate frameworks
☒ Dapper ☐ NHibernate ☒ Entity Framework Core
☒ Hibernate ☐ MyBatis ☐ EclipseLink

5. Run

Run the advisor

Result

Selected: Dapper (1 framework allowed for the whole workload)

Query	Framework	Mean time (ms)	Allocated (KB)	Relative
CustomerByCity	Dapper	0.41	12	1.00 assigned
	EF Core	0.63	41	1.54
	Hibernate	0.71	64	1.73
OrdersOfCustomer	EF Core	1.20	96	1.00
	Dapper	1.25	30	1.04 assigned
	Hibernate	1.42	118	1.18
Total (assigned)		1.66	42	

Variants of the same workload

Variant	Time (ms)	Memory (KB)
all Dapper	1.66	42
all EF Core	1.83	137
all Hibernate	2.13	182
heuristic (fastest per query)	1.61	108
ILP (selected)	1.66	42

Obrázek 5: Advisor: vstup s vahami dotazů a omezeními, výsledek jako srovnání všech změřených kandidátů s vyznačeným přiřazením a tabulka variant podle F15.

3 Platformy a technologie

Nástroj je .NET 10 solution; všechny verze závislostí jsou zafixované a spravované centrálně na jediném místě repozitáře.

- **Backend:** C#, ASP.NET Core (REST API, OpenAPI dokument), Roslyn (čtení C# entit a LINQ, kompilační stupeň ověření), `Microsoft.SqlServer.TransactSql.ScriptDom` (parsování T-SQL). Platforma, REST API i Roslyn jsou zděděné z prototypu, který běžel na .NET 8. Verzi jsme změnili na aktuální .NET 10. ScriptDom čte holé SQL Dapperu a mapperů MyBatis.
- **Překládané .NET frameworky:** Dapper 2.1.79, NHibernate 5.7.0, Entity Framework Core 10.0.10. Frameworky jsou zděděné z prototypu, zafixovali jsme jejich verze odpovídající .NET 10.
- **Java ekosystém (cíl 2, verze zafixované pro projekt):** JDK 25 (LTS), Jakarta Persistence 3.2, Hibernate ORM 7.4.5, EclipseLink 5.0.0, MyBatis 3.5.19, ovladač `mssql-jdbc`; sestavení Maven; JVM žije jen v kontejneru testovací sady a na CI runneru. Frameworky předpokládá zadání, verze JDK 25 je jako LTS protějšek .NET 10, u Hibernate a EclipseLink zvolené implementace stejné verze Jakarta Persistence a MyBatis 3.5.19 jako poslední vydání řady 3.5.
- **Databáze:** SQL Server 2022, vzorová databáze WideWorldImporters; metadata z pohledů `sys.*`. Databáze i vzorová data jsou zděděné z prototypu, kde nad nimi měří Advisor.
- **Optimalizace:** GLPK (GNU Linear Programming Kit) volaný z C wrapperu přes P/Invoke. Zděděné z prototypu.
- **Frontend:** ručně psané statické HTML, nativní ES moduly a CSS, vendorované Pico CSS a highlight.js. Prototyp měl frontend v Angularu s commitnutým zastaralým buildem. Statické stránky nepotřebují build a nasazení nezávisí na npm ekosystému. Pokud se při používání ukáže nedostatečným, přebuduje se frontend do některého JavaScriptového frameworku.
- **Testy a infrastruktura:** xUnit v3, GitHub Actions (CI), Docker a docker compose (aplikace, databáze i testovací sada jedním příkazem). Testovací sada, CI i kontejnerová konfigurace aplikace s databází jsou zděděné z prototypu. xUnit ve verzi 3 umožňuje dynamické přeskakování testů závislých na databázi s uvedeným důvodem.

Seznam není definitivní – konkrétní volby javovské strany (způsob parsování Java kódu, sestavovací nástroj, tvar testovacího harnessu) určí analýza a samostatná rozhodnutí.

4 Vyhodnocení

Vyhodnocení má dvě roviny: průběžnou kontrolu kvality nástroje a experimenty, které ověřují hlavní cíle projektu.

Kvalita nástroje. Generované artefakty se ověřují ve více stupních: (1) tvar výstupu a strukturální úplnost, (2) kompilovatelnost nebo syntaktická validita, (3) přijetí cílovým frameworkem a (4) u vybraných scénářů běh proti databázi. Pro Java frameworky vznikne analogická testovací infrastruktura v rozsahu podporované podmnožiny. Požadavky F12–F13 k tomu přidávají spustitelnou sadu a diferenční ověření výsledků dotazů nad stejnými daty.

Experimenty projektu. Jádrem experimentálního vyhodnocení je cross-language ověření: překlady .NET→Java, Java→.NET a Java→Java, jejich kompilovatelnost, spustitelnost, funkční ekvivalence a úplnost přenesených podporovaných mapovacích vlastností. Tyto experimenty mají ověřit, že mezireprezentace a překladový přístup nejsou vázány na jediný programovací jazyk a že rozšířený proof of concept je prakticky použitelný pro reprezentativní scénáře.

Navazující LLM experimenty. Projekt připraví testovací data a měřicí infrastrukturu tak, aby bylo možné deterministický pravidlový překlad porovnat se stejnými vstupy zpracovanými pomocí velkých jazykových modelů. Plné zero-shot, few-shot, RAG, agentní a ablační experimenty budou rozvedeny v navazující diplomové práci řešitele; nejsou podmínkou dokončení tohoto projektu.

Advisor a optimalizace. Existující Advisor a ILP optimalizace představují sekundární experimentální větev. Pokud to dovolí časový rozsah projektu, budou základně rozšířeny o nové frameworky a porovnány s jednoduchými baseline strategiemi. Dokončení hlavního cross-language cíle má před touto částí prioritu.

Výsledky experimentů provedených v rámci projektu budou předloženy u obhajoby spolu s naměřenými ukazateli kvality nástroje. Zvláštní důraz bude kladen na ověření cross-language použitelnosti přístupu rozšířením o Java frameworky, praktickou použitelnost rozšířené proof of concept implementace a reprodukovatelné experimentální ověření. Tyto výsledky mají sloužit jako podklad pro vědeckou publikaci, přednostně v odborném vědeckém časopise, a jako deterministická baseline pro následné plnější porovnání s LLM-based metodami.

5 Rizika

Vstup javovské strany do řešení. Zásadní nejistota cíle 2: buď javovské wrappery čtou a píší javovský kód vlastními parsery v C# (jednodušší nasazení, ale žádné ověření kompilací a přijetím frameworku), nebo k řešení přibude javovská komponenta s JVM (plné ověřovací stupně a diferenční běhy, ale druhý runtime v kontejnerové konfiguraci). Riziko se řeší jako první rozhodnutí cíle 2, s vědomím, že diferenční ověření (F13) JVM s JDBC připojením vyžaduje v každém případě – varianty se liší tím, kde JVM stojí, ne tím, jestli existuje.

Parsování javovských jazyků bez referenčních parserů. Jediný úplný parser HQL/JPQL žije uvnitř příslušného frameworku a dynamické SQL MyBatisu je vlastní XML dialekt. Zmírnění: stejný postup, jaký se osvědčil u HQL v cíli 1 – vlastní parser podmnožiny, kterou mezireprezentace unese, držený round-tripovými testy proti vlastnímu builderu; konstrukce mimo podmnožinu končí definovaným záznamem, ne tichým vynecháním.

SQL dialekty. Ručně psané SQL v MyBatis mapperech bývá psané pro jiné databáze než SQL Server a stávající T-SQL gramatika ho nepřečte. Zmírnění: rozsah se drží na referenčním SQL Serveru (jediný nárokový dialekt); co gramatika nepřečte, se hlásí záznamem o neúplnosti.

Sémantické rozdíly frameworků. Konstrukce, kterou cíl nevyjádří, nesmí zmizet potichu ani změnit význam. Mechanismus je hotový a ověřený z cíle 1: deskriptor cílového frameworku deklaruje, co framework nevyjádří, ztráty se hlásí mechanicky a dotaz s jinou množinou řádků se nevydá; javovské frameworky dostanou vlastní deskriptory. Zbytkové riziko je pracnost analýzy každého frameworku proti zafixované verzi.

Metodologie měření pro Javu. Advisor dnes měří .NET kód kompilovaný Roslynem v procesu; pro Javu je třeba najít srovnatelné běhové prostředí a vyrovnat se s rozdíly JIT, startovní režie a správy paměti. Zmírnění:

analýza prostředí je samostatný krok harmonogramu; iterační politika měření bude vyslovena a odůvodněna jako součást metodologie, aby čísla byla obhajitelná (T7). S tím souvisí i dosud nevyřešená izolace spouštění měřeného kódu (S4) – do doby rozhodnutí platí předpoklad nasazení v důvěryhodné síti a oblast je výslovně vyňata ze záruk.

Pracnost a čas. Tři nové frameworky v reprezentativním rozsahu společné mezireprezentace představují objemné zadání. Zmírnění: cíle jsou seřazené tak, aby jádro zadání (F7–F10) šlo před experimenty; mezireprezentace i slovníky jsou na Javu připravené z cíle 1; rozsah experimentů je odstupňovaný (viz kapitolu 4) a plné LLM experimenty nese až diplomová práce. Průběh je průběžně viditelný v repozitáři, takže případný skluz je vidět včas a lze ho řešit se supervizorem.

6 Milníky / Výstupy

Projekt se odevzdává jako veřejný repozitář na GitHubu [5] obsahující zdrojové kódy, kontejnerovou konfiguraci (aplikace, databáze i testovací sada jedním příkazem), testovací sadu a dokumentaci: průběžně udržovaný popis architektury, číslovaná návrhová rozhodnutí, seznam otevřených položek a explicitní hranici záruk každého vydání. Průběh prací a podíl řešitele je zřejmý z commit historie a tagů vydání.

- **Milník 1 – dokončená .NET část:** složené klíče, vztahy, katalogové doplnění, diagnostika, úplná dotazová matice tří .NET frameworků, čtyři stupně ověření.
- **Milník 2 – specifikace a analýza:** tento dokument; rozhodnutí o vstupu javovské strany a analýza běhového prostředí pro Javu.
- **Milník 3 – Java frameworky a cross-language ověření:** parsers a buildery Hibernate, MyBatis a Eclipse-Link, překlad mezi .NET a Javou a ověření použitelnosti společné mezireprezentace napříč oběma ekosystémy.
- **Milník 4 – testovací sady a experimenty:** Java testovací infrastruktura s diferenčním ověřením (F12–F13), zobrazení mezireprezentace, cross-language matice T2 s metrikami T3 a příprava infrastruktury pro navazující LLM experimenty; základní srovnání Advisoru (T7) podle časových možností.
- **Milník 5 – odevzdání:** cross-language experimentální výsledky, finální dokumentace, odevzdání a příprava obhajoby s živou ukázkou nástroje.

Komisi bude u obhajoby předvedena běžící instance (docker compose) na překladu reálných artefaktů mezi .NET a Javou, reprodukce testovací sady jedním příkazem a výsledky experimentů. Dílčí demonstrace supervizorovi a konzultantovi probíhají při uzavírání milníků.

7 Harmonogram

Milník / výstup	Aktivita	Časový plán	MD
1. Dokončení .NET části	seznámení, audit prototypu, dokumentační systém klíče, vztahy, katalog, diagnostika, dotazová matice, ověření; vydání 1.2.0	červenec	4
		červenec–srpen	9
2. Specifikace a analýza	specifikace; rozhodnutí o javovské straně, analýza běhového prostředí	září	5
3. Javovské frameworky	Hibernate (parser + builder) MyBatis EclipseLink, cross-ecosystem testy	září	4
		září–říjen	3
		září–říjen	3
4. Testy a experimenty	javovská testovací sada, diferenční ověření (F12–F13), zobrazení mezireprezentace	říjen–listopad	5
5. Odevzdání	experimenty (T2, T3, T7), dokumentace, odevzdání	listopad	5

8 Forma spolupráce

Projekt řeší jediný řešitel v rámci výzkumné skupiny Adaptive Data Management (ADaM) Research Group. Supervizorem projektu je Pavel Koupil, který je autorem zadání a odborným garantem doménové části, zejména návrhu mezireprezentace, překladových pravidel, návaznosti na související projekty a metodologie experimentů. Současně dohlíží na průběh projektu, softwarově-inženýrskou kvalitu a plnění harmonogramu.

Řešitel se pravidelně (typicky jednou za týden) setkává se supervizorem k diskusi nad provedenými změnami, návrhovými rozhodnutími a dalším plánem vývoje. Návrhová rozhodnutí a průběh implementace jsou průběžně dokumentovány v repozitáři.

Vzhledem k návaznosti na další projekty výzkumné skupiny se počítá také s příležitostnou komunikací s jejich autory. V oblasti univerzálního objektového mapování může řešitel konzultovat zejména s Martinem Čorovčákem otázky související s reprezentací a migrací mapování mezi relačními, grafovými a dokumentovými systémy. Pro synchronizaci s projektem mm-mapsearch a souvisejícími pracemi může podle potřeby komunikovat s Bedřichem Mazourkem nebo Jáchymem Bártíkem. V době této spolupráce již mohou mít tito studenti své projekty dokončené nebo obhájené; jejich role je proto chápána především jako konzultace s autory předchozích řešení a předávání zkušeností.

Projekt zároveň předpokládá spolupráci s novými členy týmu, kteří se budou zapojovat do navazujících témat datové migrace, mapování a experimentálního vyhodnocení. Podle potřeby budou jednotlivé otázky konzultovány také s Irenou Holubovou nebo Michalem Kopeckým, zejména v souvislosti s vlastnostmi relačního datového modelu, databázových schémat a sémantikou relačních dat.

Reference

- [1] ADAPTIVE DATA MANAGEMENT RESEARCH GROUP. *Adaptive Data Management Research Group*. Dostupné z: <https://www.cnaip.cz/en/entity/adaptive-data-management-research-group>
- [2] KOUPIL, Pavel et al. *mm-mapsearch*. Stránka nástroje. Dostupné z: <https://www.ksi.mff.cuni.cz/~koupil/software/mm-mapsearch/>
- [3] KOUPIL, Pavel. *ORMorpher*. Stránka nástroje. Dostupné z: <https://www.ksi.mff.cuni.cz/~koupil/ormorpher/index.html>
- [4] ABRAHÁM, Milan. *orm-convertor* (ORMorpher). Repozitář prototypu a text diplomové práce. Dostupné z: <https://github.com/milan252525/orm-convertor>
- [5] ŠTOCHEL, Miroslav. *orm-convertor* (ORMConvertor). Repozitář projektu. Dostupné z: <https://github.com/MiraStochel/orm-convertor>